

Module 17) Javascript For Full Stack Assignment

Question 1: What is JavaScript? Explain the role of JavaScript in web development.

Ans:- JavaScript is a high-level, interpreted programming language used to make web pages interactive. It runs in the browser and can update content dynamically without reloading the page. It is mainly used for client-side scripting like form validation, animations, and event handling. In full stack development, it works with HTML (structure) and CSS (style) and JavaScript (Behavior).

Question 2: How is JavaScript different from other programming languages like Python or Java?

Ans:- JavaScript is mainly used for web development and runs in browsers, while Python and Java are general-purpose languages. Java is compiled, while JavaScript and Python are interpreted. JavaScript supports event-driven programming, which is useful for UI interactions. Python is simpler for backend and data tasks.

Question 3: Discuss the use of <script> tag in html. How can you link an external js file to an html document ?

Ans:- <script> is used to include JavaScript in HTML. It can be inline or external. To link an external JS file, use <script src="script.js"></script>. External files help in code reusability and cleaner structure.

● Variables and Data Types

Question 1: What are variables in JavaScript? How do you declare a variable using var, let, and const?

Ans:- Variables are used to store data values in JavaScript. var is function-scoped and can be redeclared. let is block-scoped and cannot be redeclared in the same scope. const is also block-scoped but its value cannot be changed after assignment. Example: let x = 10; var x = 10; const y = 20;.

Question 2: Explain the different data types in JavaScript. Provide examples for each.

Ans:- JavaScript has primitive and non-primitive data types. Primitive types, These are **basic, single values** (immutable) include Number (10), String ("Hello"), Boolean (true), Undefined, Null, Symbol, and BigInt. Non-primitive types include Object and Array. Example: let a = 5; let b = "text"; Arrays: [1,2,3].

Question 3: What is the difference between undefined and null in JavaScript?

Ans:-

undefined	Null
Indicates a variable that has been declared	Represents the intentional absence of any

but not yet assigned a value.	object value.
Primitive data type in JavaScript.	Primitive data type in JavaScript.
Automatically assigned to variables that are declared but not initialized.	Must be explicitly assigned to a variable.
undefined == null // true (loose equality considers them equal).	undefined == null // true (loose equality considers them equal).
undefined === null // false (they are not strictly equal in type).	null === undefined // false (they are not strictly equal in type).
Used by JavaScript when a variable is not assigned a value.	Used intentionally by developers to indicate "no value" or "empty".
undefined is a global property with the value undefined .	null is a global property with the value null .
undefined values are omitted during serialization.	null values are preserved during JSON serialization (e.g., {"key": null}).

● JavaScript Operators

Question 1: What are the different types of operators in JavaScript? Explain with examples.

Ans:-JavaScript provides different types of operators to perform operations on variables and values.

Arithmetic: + - * / (e.g., 2+3=5)

Assignment: = += (e.g., x+=2)

Comparison: ==, >, < (e.g., x==5)

Logical: &&, || (e.g., true && false)

Operators are used to perform operations on variables and values. They are essential for logic building.

Question 2: What is the difference between ==and ===in JavaScript?

Ans:- == compares values only and does type conversion (loose equality). === compares both value and data type (strict equality). Example: 5 == "5" → true, but 5 === "5" → false. Always prefer === for accurate comparison.

● Control Flow (If-Else, Switch)

Question 1: What is control flow in JavaScript? Explain how if-else statements work within example.

Ans:- Control flow determines the order in which code executes. if-else is used to make decisions based on conditions. If condition is true, if block runs, otherwise else runs.

Example: if (a > 10) { console.log("Big"); } else { console.log("Small"); }

Question 2: Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

Ans:- A switch statement checks a variable against multiple cases. It is used when there are many conditions based on one value. Each case ends with break to stop execution. It is cleaner than multiple if-else for fixed values. Use switch when checking one variable with many options.

Example: `switch(day){ case 1: ... }`

- **Loops (For, While, Do-While)**

Question 1: Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

Ans:- for loop is used when number of iterations is known. while loop runs while condition is true. do-while runs at least once even if condition is false.

Example:

```
for(let i=0;i<5;i++){  
while(i<5){  
do {} while(i<5);
```

- **Question 2: What is the difference between a while loop and a do-while loop?**

Ans:- while checks condition before execution, so it may not run at all. do-while executes first and checks condition later. So it always runs at least once. Example: do-while is useful for menus.

- **Functions**

- **Question 1: What are functions in JavaScript? Explain the syntax for declaring and calling a function.**

Ans:- Functions are reusable blocks of code used to perform tasks. They help in reducing code repetition. Syntax:

Syntax: `function name() {}`

Call: `name();`

Example: `function add(a,b){ return a+b; }
function greet() { console.log("Hello"); }
greet();`

- **Question 2: What is the difference between a function declaration and a function expression?**

Ans:- Function declaration defines a function using function keyword. Function expression stores function in a variable. Example:

`function a() {}` vs `let b = function() {}`.

Expressions are not hoisted like declarations.

Function Declaration

```
function add(a, b) {  
  return a + b;  
}
```

Function Expression

```
const add = function(a, b) {  
  return a + b;  
};
```

• Question 3: Discuss the concept of parameters and return values in functions.

Ans:- Parameters are inputs passed to a function. Return value is the output returned from function. Example:

```
function add(a,b){ return a+b; }
```

This makes functions dynamic and reusable.

• Arrays

• Question 1: What is an array in JavaScript? How do you declare and initialize an array?

Ans:- Array is a collection of multiple values in a single variable. It is declared using square brackets. Example:

```
let arr = [1,2,3];
```

Arrays can store different data types.

• Question 2: Explain the methods push(), pop(), shift(), and unshift() used in arrays.

Ans:- push() adds element at end.

pop() removes last element.

shift() removes first element.

unshift() adds element at beginning.

Example: arr.push(4);

• Objects

• Question 1: What is an object in JavaScript? How are objects different from arrays?

Ans:- Objects store data in key-value pairs. Arrays store values in index form.

Example:

```
let obj = {name:"Sakshi"}.
```

Objects are used for structured data, arrays for lists.

- **Question 2: Explain how to access and update object properties using dot notation and bracket notation.**

Ans: Dot notation: obj.name

Bracket notation: obj["name"]

Update: obj.name = "New";

Bracket is useful when key is dynamic.

- **JavaScript Events**

- **Question 1: What are JavaScript events? Explain the role of event listeners.**

Ans:- Events are user actions like click, hover, or keypress. Event listeners detect these actions and run functions. They make web pages interactive. Example: clicking a button triggers an action.

- **Question 2: How does the addEventListener() method work in JavaScript? Provide an example**

Ans:- It attaches an event to an element without overwriting others.

Example:

```
button.addEventListener("click", function(){  
    alert("Clicked");  
});
```

- **DOM Manipulation**

- **Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?**

Ans:- DOM is a tree structure of HTML elements. JavaScript uses DOM to access and modify content, structure, and style. Example: changing text or adding elements dynamically.

- **Question 2: Explain the methods getElementById(), getElementsByClassName(), and querySelector() used to select elements from the DOM.**

Ans:- getElementById() selects by id.

getElementsByClassName() selects multiple elements.

querySelector() selects first matching element.

These help in targeting elements for manipulation.

- **JavaScript Timing Events (setTimeout, setInterval)**

- **Question 1: Explain the setTimeout() and setInterval() functions in JavaScript. How are they used for timing events?**

Ans:- setTimeout() runs a function after a delay (once).

setInterval() runs repeatedly after a fixed time.

Used for animations, alerts, timers.

- **Question 2: Provide an example of how to use setTimeout() to delay an action by 2 seconds.**

Ans:- `setTimeout(function(){
 console.log("Hello after 2 seconds");
}, 2000);`

- **JavaScript Error Handling**

- **Question 1: What is error handling in JavaScript? Explain the try, catch, and finally blocks with an example.**

Ans:- Error handling prevents program crash.

try contains risky code.

catch handles error.

finally runs always.

Example:

```
try { x=y; } catch(e){ console.log(e); }
```

- **Question 2: Why is error handling important in JavaScript applications?**

Ans:- It improves program stability and user experience. Prevents app crashes. Helps debug issues easily. Ensures smooth execution even when errors occur.